

AEL-2

Tree Data Structure

Definition and Types

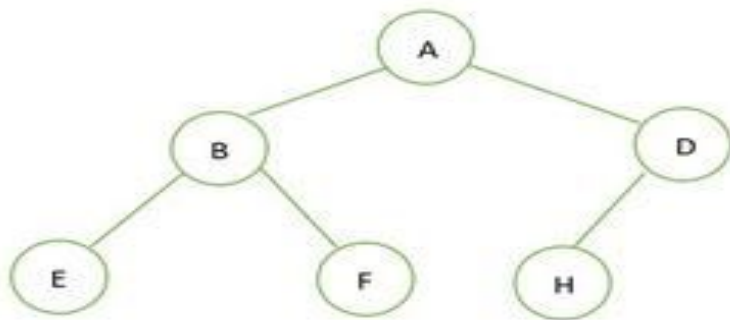
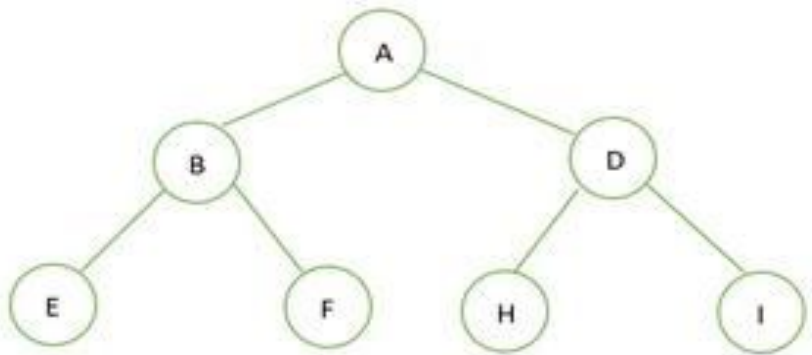
A **tree data structure** is a hierarchical structure that is used to represent and organize data in a way that is easy to navigate and search.

Tree can be of three types : Binary, Ternary and N-ary or Generic Tree.

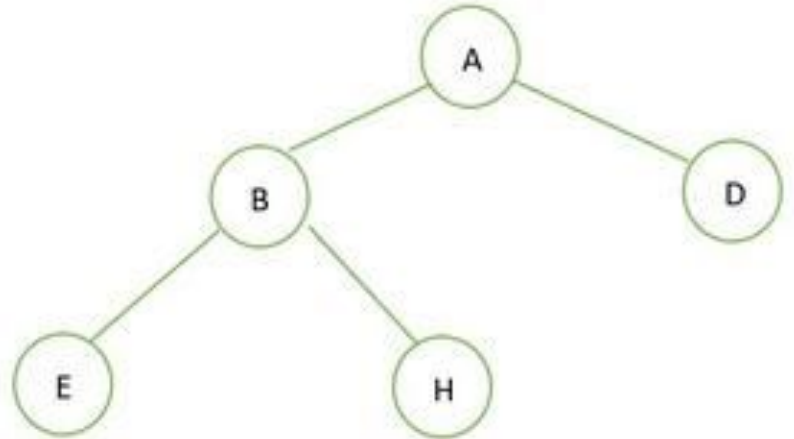
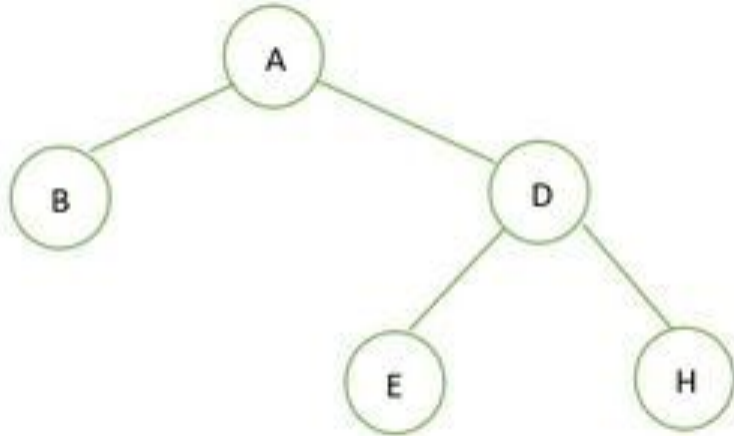
But normally when we ask about types of Tree, we mean types of Binary Tree.

- a) Full : A full binary tree is a binary tree with either zero or two child nodes for each node.
- b) Complete : A complete binary tree is a special type of binary tree where all the levels of the tree are filled completely except the lowest level nodes which are filled from as left as possible.
- c) Perfect : A binary tree of height 'h' having the maximum number of nodes is a perfect binary tree.

Perfect vs Complete



Full and Complete Binary Tree



Tree Traversal

The most famous two tree traversal technique is DFS and BFS.

DFS : Depth-first search is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking.

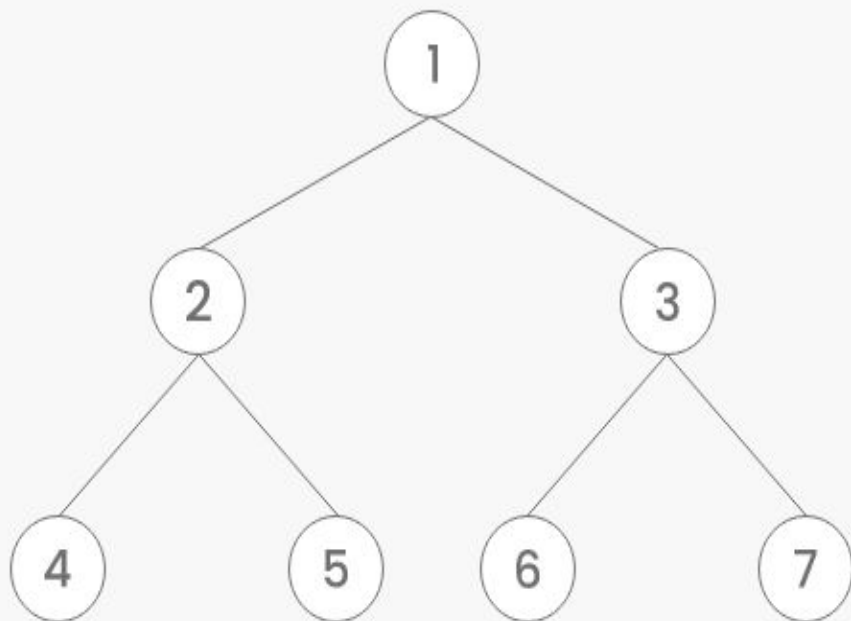
BFS : Breadth First Search or Level Order Traversal technique is defined as a method to traverse a Tree such that all nodes present in the same level are traversed completely before traversing the next level.

There are three types of BST traversal : In order, Pre order and Post order.

For In order algorithm is,

- Traverse left subtree.
- Visit the root and print the data.
- Traverse the right subtree.

Tree Traversal Techniques



Inorder Traversal

4	2	5	1	6	3	7
---	---	---	---	---	---	---

Preorder Traversal

1	2	4	5	3	6	7
---	---	---	---	---	---	---

Postorder Traversal

4	5	2	6	7	3	1
---	---	---	---	---	---	---

Tree Properties

- **Number of edges:** An edge can be defined as the connection between two nodes. If a tree has N nodes then it will have $(N-1)$ edges. There is only one path from each node to any other node of the tree.
- **Depth of a node:** The depth of a node is defined as the length of the path from the root to that node. Each edge adds 1 unit of length to the path. So, it can also be defined as the number of edges in the path from the root of the tree to the node.
- **Height of a node:** The height of a node can be defined as the length of the longest path from the node to a leaf node of the tree.
- **Height of the Tree:** The height of a tree is the length of the longest path from the root of the tree to a leaf node of the tree.
- **Degree of a Node:** The total count of subtrees attached to that node is called the degree of the node. The degree of a leaf node must be **0**. **The degree of a tree is the maximum degree of a node among all the nodes in the tree.**

Application of Trees

- **File System:** This allows for efficient navigation and organization of files.
- **Data Compression:** Huffman coding is a popular technique for data compression that involves constructing a binary tree where the leaves represent characters and their frequency of occurrence. The resulting tree is used to encode the data in a way that minimizes the amount of storage required.
- **Compiler Design:** In compiler design, a syntax tree is used to represent the structure of a program.
- **Database Indexing:** B-trees and other tree structures are used in database indexing to efficiently search for and retrieve data.

Huffman Coding

Huffman coding is a lossless data compression algorithm.

- The idea is to assign variable-length codes to input characters, lengths of the codes are based on the frequencies of characters. The greedy idea is to assign the least length code to the most frequent character.
- The variable-length codes assigned to input characters are Prefix Codes, means the codes (bit sequences) are assigned in such a way that the code assigned to one character is not the prefix of code assigned to any other character. This is how Huffman Coding makes sure that there is no ambiguity when decoding the generated bitstream.

Let us understand prefix codes with a counter example. Let there be four characters a, b, c and d, and their corresponding variable length codes be 00, 01, 0 and 1. This coding leads to ambiguity because code assigned to c is the prefix of codes assigned to a and b. If the compressed bit stream is 0001, the de-compressed output may be "cccd" or "ccb" or "acd" or "ab".'

Steps to build Huffman Tree

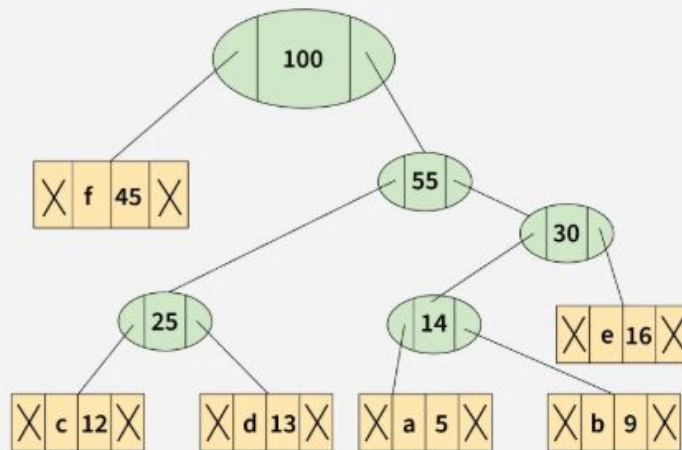
- Create a leaf node for each unique character and build a min heap of all leaf nodes (Min Heap is used as a priority queue. The value of frequency field is used to compare two nodes in min heap. Initially, the least frequent character is at root)
- Extract two nodes with the minimum frequency from the min heap.
- Create a new internal node with a frequency equal to the sum of the two nodes frequencies. Make the first extracted node as its left child and the other extracted node as its right child. Add this node to the min heap.
- Repeat steps#2 and #3 until the heap contains only one node. The remaining node is the root node and the tree is complete.

Complete Process of Huffman Coding

Input: $s = \text{"abcdef"}$, $f[] = [5, 9, 12, 13, 16, 45]$

Output: $[0, 100, 101, 1100, 1101, 111]$

Explanation: Huffman Tree would be



Huffman Codes would be:

$f: 0$

$c: 100$

$d: 101$

$a: 1100$

$b: 1101$

$e: 111$

These codes are generated using Huffman Tree by considering 0 value for left child and 1 for right. We get codes by traversing from root to every character and using 0 or 1 for every edge along the path.

File System

- Top right image is simple node can be used to implement today's SOTA file systems.
- The bottom right image is more complex with enum type data. A value of type FileSystemNode can be either a Directory or a File, but never both at the same time.

Problem: Actual data can only be stored in leaf nodes.

```
struct SimpleTreeNode<T> {  
    data: T,  
    children: Vec<SimpleTreeNode<T>>,  
}
```

```
enum FileSystemNode {  
    Directory {  
        name: String,  
        children: Vec<FileSystemNode>,  
    },  
    File {  
        name: String,  
        data: Vec<u8>  
    },  
}
```

Intuition

We can see that very practically using standard linux / programming tools. While you can use `cat` to show the data of a file, it won't work for directories. And the other way around, you can't use `cd` to navigate into a file or `ls` to list its children. Another example can be found in your IDE: Double-clicking on a folder doesn't show content in the main editor area and you can't create sub-files of a file through a context menu entry.

If you're thinking of files and folders using their actual analog equivalents (sheets of paper in a folder), opening a sheet of paper (`cd` equivalent) doesn't make any sense. On the other hand, you could probably argue that paper folders can actually contain data themselves if you write it on the cover, but that's not a solution for larger amounts of data.

The fact that inner nodes in a file system hierarchy cannot hold data is limiting. In many use cases, it'd be natural to have nodes that can hold data AND have children. Because file systems don't allow this, we often use workarounds.

Html Websites

When an HTTP client (generally a web browser) requests a URL that points to a directory structure instead of an actual web page within the directory structure, the web server will generally serve a default page, which is often referred to as a main or “index” page. A common filename for such a page is `index.html`, but most modern HTTP servers offer a configurable list of filenames that the server can use as an index.

In our case, requesting `http://localhost:8000/blog/posts` will show the `index.html` file in the `posts` folder. The `index.html` file contains the data that logically belongs to its parent folder (`posts` in this case).

```
blog/  
  posts/  
    index.html  
    hello-world.html  
    file-system-tree.html  
  index.html  
  about.html
```

Comparison of two Updated Tree Structure

Even though Updated FileSystemNode has all the functionality we have, such as creating file nodes that has both content and files, the best implementation is SimpleTreeNode. Since, with simple use of template we can make SOTA file systems both simpler and general.

```
struct FileSystemNode {  
    name: String,  
    children: Vec<FileSystemNode>,  
    content: Vec<u8>,  
}
```

```
type FileSystemNode2 = SimpleTreeNode<(String, Vec<u8>>>;
```

BST

Binary Search Tree is a node-based binary tree data structure that has the following properties:

- The left subtree of a node contains only nodes with keys lesser than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- The left and right subtree each must also be a binary search tree.

Problem

Given Binary Search Tree.

The task is to find sum of all elements smaller than and equal to Kth smallest element.

Solution : Go for Inorder traversal (it would make our traversal sorted) and count if we have already visited k nodes.

Input : K = 3



Output : 17

Explanation : Kth smallest element is 8 so sum of all element smaller than or equal to 8 are
 $2 + 7 + 8$

Continued

But the previous solution takes $O(n)$ complexity. We want to do it in $O(h)$ complexity where h means height of the binary tree.

The right side pseudo code snippet can solve the problem in $O(h)$.

BST Node contain to extra fields : lcount , Sum

For each Node of BST

lCount : store how many left child it has

Sum : store sum of all left child it has

Find Kth smallest element

[temp_sum store sum of all element less than equal to K]

ksmallestElementSumRec(root, K, temp_sum)

```
IF root -> lCount == K + 1
```

```
    temp_sum += root->data + root->sum;
```

```
    break;
```

```
ELSE
```

```
    IF k > root->lCount    // Goto right sub-tree
```

```
        temp_sum += root->data + root-> sum;
```

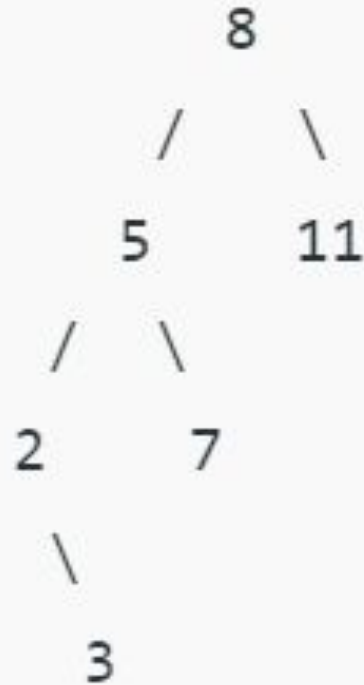
```
        ksmallestElementSumRec(root->right, K-root->lcount+1, temp_sum)
```

```
    ELSE
```

```
        // Goto left sub-tree
```

```
        ksmallestElementSumRec( root->left, K, temp_sum)
```

Another Test Case with $K = 3$



Segment Tree

A Segment Tree is a data structure that **stores information about a range of elements in its nodes.**

It also allows users to **modify the array** and **perform range queries in smaller complexity.**

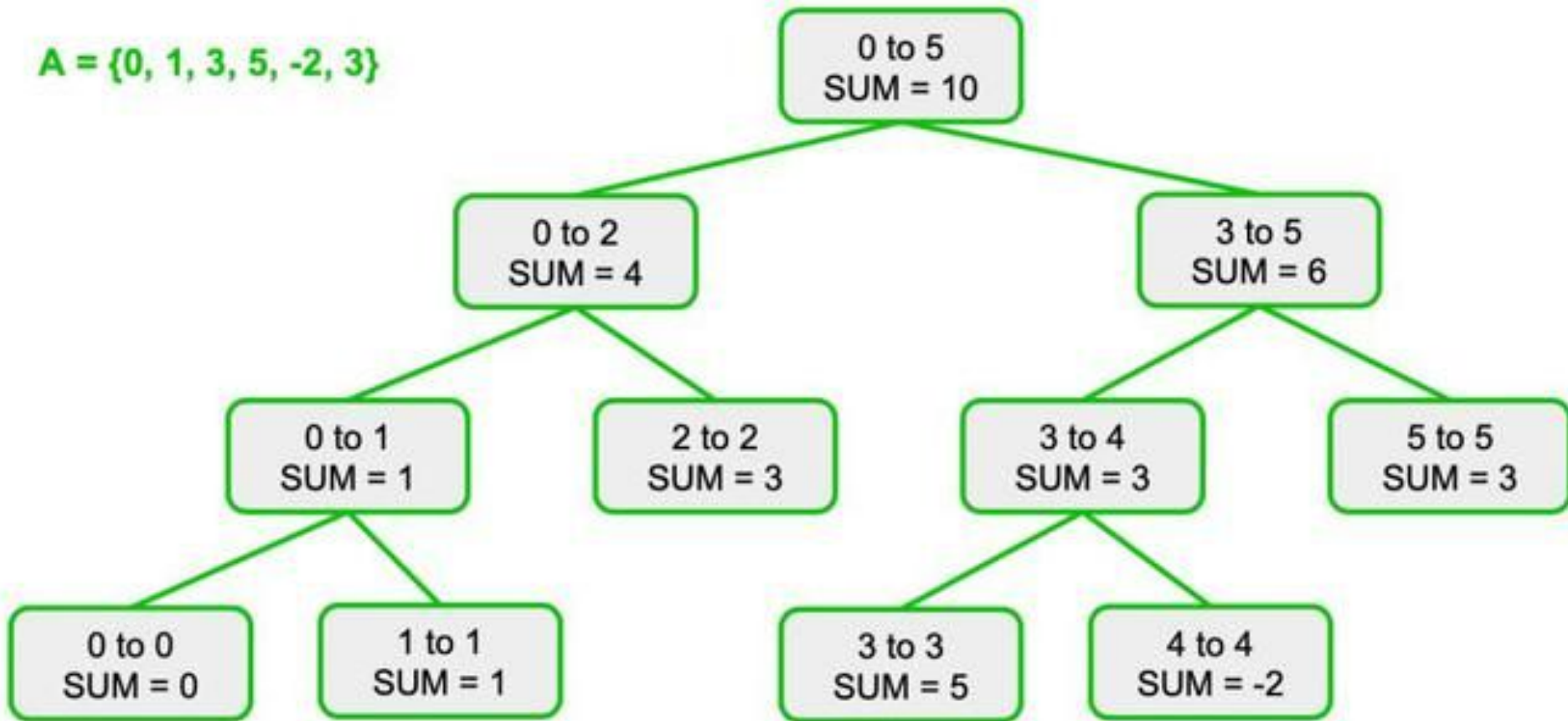
For example, we can perform a range summation of an array between the range L to R while also modifying the array from range **L** to **R** all in **$\log(N)$** time complexity.

Some of the examples of operations are:

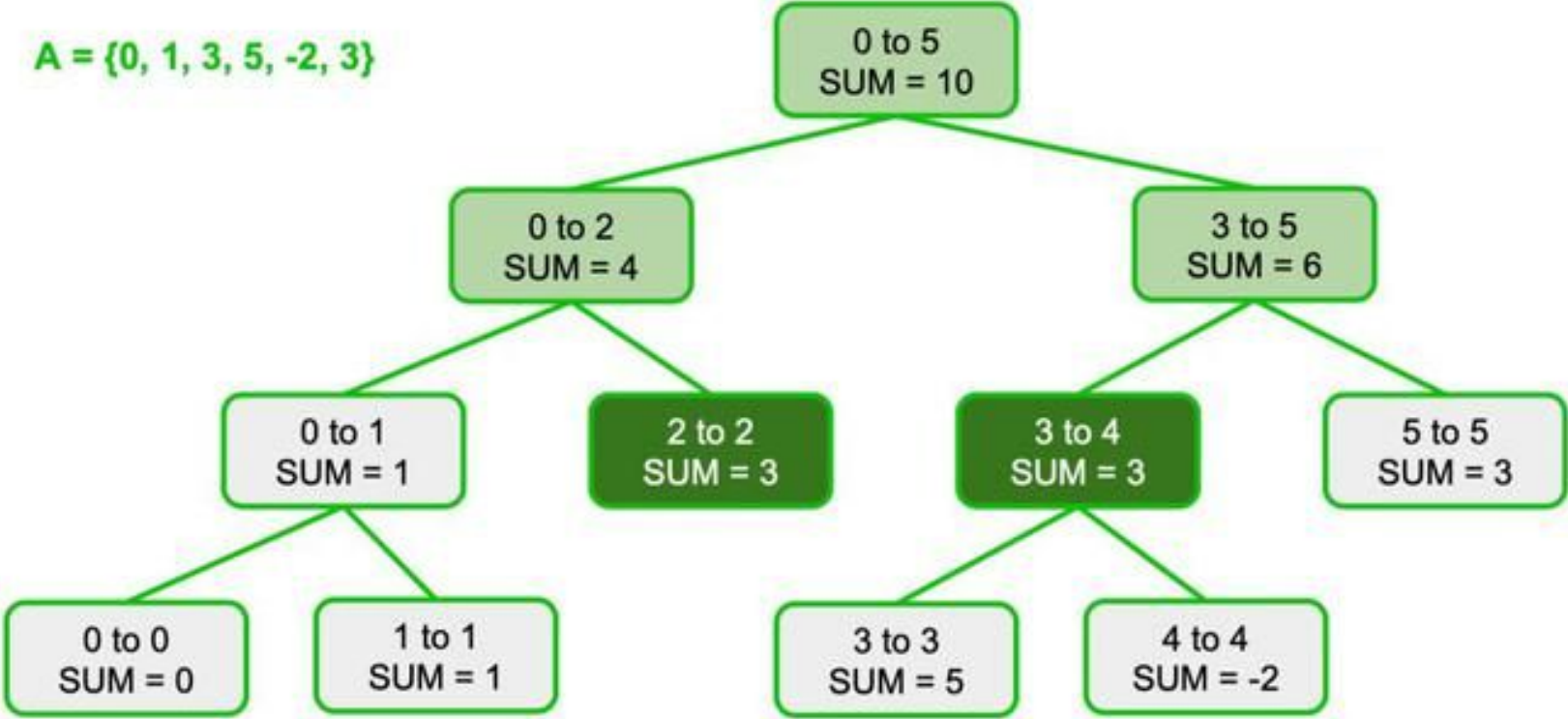
- Addition/Subtraction
- Maximum/Minimum
- GCD/LCM
- AND/OR/XOR

Building Segment Tree $O(n \log n)$

$A = \{0, 1, 3, 5, -2, 3\}$



Range query $O(\log n)$



For sum from range 2 to 4 nodes that are highlighted are traversed. The answer is collected from the dark highlighted nodes.